

3.12 [20] <§3.3>Using a table similar to that shown in Figure 3.6, calculate the product of the octal unsigned 6-bit integers 62 and 12 using the hardware described in Figure 3.3. You should show the contents of each register on each step

62 base 8 $\rightarrow$ 110 010

12 base 8 $\rightarrow$ 001 010

Iteration	Step	Multiplier	Multiplicand	Product
0	Initial Values	001010	000000110010	000000000000
1	Shift left mcand	001010	000001100100	000000000000
1	Shift right mcand	000101	000001100100	000000000000
2	Product = Prod + Mcand	000101	000001100100	000001100100
2	Shift left mcand	000101	000011001000	000001100100
2	Shift right mult	000010	000011001000	000001100100
3	Shift left mcand	000010	000110010000	000001100100
3	Shift right mult	000001	000110010000	000001100100
4	Prod + mult = product	000001	000110010000	00011110100
4	Shift left mcand	000001	001100100000	00011110100
4	Shift right mult	000000	001100100000	00011110100

Post-table product = 00011110100 base 2 = **764 base 8**

3.17 [20] <§3.3>As discussed in the text, one possible performance enhancement is to do a shift and add instead of an actual multiplication. Since 9×6 , for example, can be written $(2 \times 2 \times 2 + 1) \times 6$, we can calculate 9×6 by shifting 6 to the left three times and then adding 6 to that result. Show the best way to calculate $0 \times 33 \times 0 \times 55$ using shifts and adds/subtracts. Assume both inputs are 8-bit unsigned integers.

$$33 = 00100001 = 2^5 + 2^0 = 32 + 1$$

$$55 = 00110111 = 2^5 + 2^4 + 2^2 + 2^1 + 2^0 = 32 + 16 + 4 + 2 + 1$$

$$33 \times 55 \rightarrow 33 \times (32 + 16 + 4 + 2 + 1) \text{ or } (2^5 + 2^4 + 2^2 + 2^1 + 2^0)$$

$$33 \times 2^5 \rightarrow \text{Shift 33 left by 5 bits} = 0010000100000 = 1056$$

$$33 \times 2^4 \rightarrow \text{Shift 33 left by 4 bits} = 001000010000 = 528$$

$$33 \times 2^2 \rightarrow \text{Shift 33 left by 2 bits} = 0010000100 = 132$$

$$33 \times 2^1 \rightarrow \text{Shift 33 left by 1 bit} = 001000010 = 66$$

$33 \times 2^0 \rightarrow \text{Shift } 33 \text{ left by } 0 \text{ bits} = 00100001 = 33$
 $1056 + 528 + 132 + 66 + 33 = 1815$
 $1815 \times 0 = 0$

3.18 [20] <§3.4> Using a table similar to that shown in Figure 3.10, calculate 74 divided by 21 using the hardware described in Figure 3.8. You should show the contents of each register on each step. Assume both inputs are unsigned 6-bit integers.

$74 = 000000100101$
 $21 = 010101000000$

Iteration	Step	Quotient	Divisor	Remainder
0	Initial values	000000	010101000000	000001001010
1	Rem = Rem - Div	000000	010101000000	101100001010
1	Rem < 0 $\rightarrow$ +Div, LSL Q, Q0 = 0	000000	010101000000	000001001010
1	Shift div right	000000	001010100000	000001001010
2	Rem = Rem - Div	000000	001010100000	110110101010
2	Rem < 0 $\rightarrow$ +Div, LSL Q, Q0=0	000000	001010100000	000001001010
2	Shift div right	000000	000101010000	000001001010
3	Rem = Rem - Div	000000	000101010000	111101111010
3	Rem < 0 $\rightarrow$ +Div, LSL Q, Q0=0	000000	000101010000	000001001010
3	Shift div right	000000	000010101000	000001001010
4	Rem = Rem - Div	000000	000010101000	111110100010
4	Rem < 0 $\rightarrow$ +Div, LSL Q, Q0=0	000000	000010101000	000001001010
4	Shift div right	000000	000001010100	000001001010
5	Rem = Rem - Div	000000	000001010100	111111101110
5	Rem < 0 $\rightarrow$ +Div, LSL Q, Q0=0	000000	000001010100	000001001010
5	Shift div right	000000	000000101010	000001001010

6	Rem = Rem - Div	000000	000000101010	0000000100000
6	Rem >= 0 → LSL Q, Q0 = 1	000001	000000101010	0000000100000
6	Shift div right	000001	000000010101	0000000100000
7	Rem = Rem - Div	000001	000000010101	000000001011
7	Rem >= 0 → LSL Q, Q0 = 1	000011	000000010101	000000001011
7	Shift div right	000011	000000001010	000000001011

Quoitent = 000011 = 3

Divisor = 001010 = 10

Remainder = 001011 = 11

Therefore, 74/21 = 3 remainder 11

3.23 [10] <§3.5>Write down the binary representation of the decimal number 63.25 assuming the IEEE 754 single precision format.

Convert to binary

$63 = 111111 = 2^0 + 2^1 + 2^2 + 2^3 + 2^4 + 2^5$

$0.25 = .01$ as $2 \cdot .25 = .5$ and $0.5 \cdot 2 = 1$

Therefore, 63.25 in binary is 111111.01

Then convert to scientific notation

111111.01 in scientific notation is $1.1111101 \cdot 2^5$

Find sign, bit, mantissa

Sign Bit: Positive, so the bit = 0

Exponent: 5, but need to add 127 (IEEE 754 format)

$5 + 127 = 132 = 10000100$

Fraction: 1111101 (decimal in scientific)

Must add 0s to make it 23 bits: 11111010000000000000000

Sign Bit = 0 | Exponent = 10000100 | Fraction= 11111010000000000000000

0 10000100 11111010000000000000000

3.24 [10] <§3.5>Write down the binary representation of the decimal number 63.25 assuming the IEEE 754 double precision format.

Double precision consists of 64 bits instead of 32 as seen previously.

63 converted to binary = 111111

0.25 coveted to binary = .01

Therefore, 63.25 in binary = 111111.01

Then convert from binary to scientific: $111111.01 = 1.1111101 \cdot 2^5$

1 01100 0100000000